Fleet Simulation Engine

Andrew Ma | Saronic Project 2 | `https://github.com/andrewfo/Sproject`

Status: all three stages (base engine, cargo and docks, history) are done. Usage is in README and docs.

Overview

I built a Rust simulator where vessels travel between ports, move cargo around, and compete for dock space. It takes JSON commands one line at a time and answers each one. It also snapshots every tick, so you can go back and ask what the world looked like at any earlier point.

Architecture

All of the simulation logic lives in the library. The CLI just does input and output, and the rest is split into modules for JSON parsing, data types, rules, and errors.

The most important decision I made was to keep committed state and planned state separate. Committed state is the world as of the last finished tick; planned state is that same world + every command accepted since then. When a tick runs, the plan gets committed, arrivals are updated, and an image is saved.

At first the problem is just moving boats forward one tick at a time. Cargo complicates it, because several commands can compete before time moves. Whether a boat is allowed to load depends on committed state, but what it can actually load depends on planned inventory.

Implementation

Cargo transfers run with a copy and only complete if the whole operation succeeds, so a failed swap can't leave half its cargo moved, and rejected cargo cant move the clock.

For dock throughput, I measure how much work it takes to get from the committed inventory to the planned one. A swap is one operation, and loading a unit and then putting it back is zero. I also realized deliveries need their own tracking, because delivered cargo disappears from inventory.

I worked in stages (base engine, then cargo and docks, then history) and didn't move on until each one passed tests and Clippy. The CLI tests cover malformed input, Windows line endings, and blank lines.

Key decisions and tradeoffs

A single world would have mixed current state up with pending changes. A command queue would have meant going back through everything to find out what cargo is available. A separate plan avoids both problems, but you have to have a second copy of the world. Full snapshots are the same, where history stays easy, but memory use increases.

For throughput, counting cargo commands would have been simpler than diffing inventories, but it would charge docks for work that later commands undo. Diffing the starting and planned inventories is better..

Dock configuration is optional so the original examples still run just fine. Unknown JSON fields get rejected, and writing JSON directly is faster than building a tree first, but then getting the order right is another challenge.

Results

The final matches the output of all three given examples. The test suite has 101 unit tests, 3 CLI tests, 11 golden-output tests, and one property test. Eight of the golden tests are edge cases I came up with on my own. The property test gives random command streams to the engine and checks that cargo isnt lost or duplicated, that planned and committed state agree after every tick, and that the stored history dosent change.

The biggest performance gain came from response formatting. It turns out that building a temporary JSON tree for every response was using almost all the time. Writing JSON directly took a 1,000-vessel, 1,000-tick run from 24 seconds to 2, and the speedup was consistently 11x across three workloads.

Limitations and lessons learned

Every response still sends back the entire world. A 1,000-vessel, 4,000-tick run used about 1 GB.

Cargo ordering is still an open question. Loading a unit and putting it back costs zero net dock operations, but it still moves that unit to the end of the dock's cargo list.

Loading cargo and then delivering it revealed a missing throughput charge. A single bad input line could collapse the whole program. Very large travel times broke the arrival-time math. Every fix for these came with a regression test.

I also learned that a passing test isn't necessarily a good test, especially since Claude wrote a lot of mine. Comparing parsed JSON couldn't catch field-order changes, and one test was literally comparing a world to itself.

I used Claude Code as a pair programmer the whole way, working in small commits and checking between steps. Its output still needed my review: I cut redundant tests and functions.

Next steps

First, I'd shrink history's memory footprint, either by sharing unchanged data between snapshots or by taking an occasional full snapshot and only keeping the changes. I'd also settle on a cargo-ordering rule.

For a real fleet, I'd prioritize recovery and predictable command handling. That means periodic checkpoints plus a log of accepted commands so the engine can replay everything since the last checkpoint, a defined ordering when multiple clients send commands at once, and request IDs so a retry cannot be applied twice.